Trabalho prático de Arquitetura de Computadores

Análise de desempenho do método de Jacobi paralelo com Linux perf

Disciplina de Arquitetura de Computadores

1 Especificação do trabalho

O trabalho consiste em uma análise experimental do desempenho de um resolvedor paralelo de sistemas lineares baseado no método de Jacobi, implementado em C com OpenMP. O trabalho deverá ser desenvolvido em **grupos de até três discentes**.

Cada grupo deverá escolher **três métricas de desempenho disponíveis no comando `perf` do Linux**, fundamentando sua escolha na Seção 3, "Perf interface", do documento `perf.pdf`. As métricas deverão permitir investigar o efeito das otimizações do compilador e das características arquiteturais das máquinas sobre a execução da aplicação.

> **É proibido alterar qualquer linha do código-fonte do resolvedor.** Não inserir instrumentação, modificar o algoritmo, as diretivas OpenMP, o critério de parada ou as estruturas de dados. As três versões serão produzidas exclusivamente pelas configurações de compilação já existentes no Makefile fornecido, que também deverá ser preservado. É permitido criar scripts externos para execução, validação dos resultados, processamento estatístico e geração dos gráficos.

O grupo deve baixar o material do link[1]. O arquivo ZIP contém **o código-fonte, o Makefile, uma instância de teste e as instâncias disponibilizadas para os experimentos**. Deverão ser utilizados os sistemas de dimensões **1000, 2000, 3000, 4000 e 5000**, com os respectivos arquivos de entrada.

Os experimentos deverão ser realizados em **duas máquinas do laboratório 306: uma nova e uma velha**. Para cada métrica, tamanho de entrada, versão compilada e máquina, executar o programa **dez vezes**, registrando os dados individuais, a média e o desvio padrão.

O uso de modelos de linguagem de grande porte (LLMs) é permitido, mas não obrigatório. O grupo deverá declarar expressamente se utilizou essas ferramentas e, em caso afirmativo, onde e como as utilizou. O uso de LLM não transfere à ferramenta a responsabilidade pelos experimentos, scripts, cálculos e conclusões, nem dispensa os discentes de compreenderem a solução entregue.

OBSERVAÇÃO: Algumas métricas precisam ser executadas como administrador da máquina, usuário `root`. Além disso, é necessário ativar o recurso de leitura das métricas com o parâmetro do kernel na linha de comando:

```
sudo sh -c "echo 2 > /proc/sys/kernel/perf_event_paranoid"
```

1.1 Avaliação e prazos

Até **15 de outubro de 2026**, os discentes deverão informar a composição do grupo, as três métricas escolhidas e o planejamento experimental, incluindo as máquinas e a quantidade de threads proposta.

O material completo deverá ser enviado até **27 de novembro de 2026**, **exclusivamente pelo SIGAA**. Não serão aceitos materiais enviados por e-mail ou entregues no dia da apresentação. A não entrega no prazo será entendida como desistência do grupo em realizar o trabalho.

[1] `http://www.dcc.ufrrj.br/~marcelo/aulas/TN712/fontes.zip`

As apresentações ocorrerão entre **7 e 11 de dezembro de 2026**, na semana que antecede a semana da optativa, conforme programação do docente. Cada discente será arguido individualmente, em um total de **seis perguntas**. Todos os integrantes deverão compreender o trabalho completo, e não apenas a parte que executaram.

Data	Atividade
15/10/2026	Informar integrantes, três métricas e planejamento experimental.
27/11/2026	Entregar fontes originais, scripts, dados e documentação pelo SIGAA.
07 a 11/12/2026	Apresentação, demonstração e arguição individual com seis perguntas.

> **Ter os executáveis apenas compilando não é suficiente para garantir a nota.** A avaliação considera a execução correta, a qualidade dos experimentos e, principalmente, a compreensão demonstrada por cada discente na arguição oral. Resultados e gráficos completos não asseguram pontuação nas perguntas que o discente não souber responder.

2 Requisitos experimentais

2.1 Três versões e otimizações de compilação

Utilizar o arquivo `main-solver-jacobi-mt.c` sem modificações e o Makefile existente no subdiretório do material fornecido. São obrigatórias as três versões abaixo:

Configuração	Executável
`-O0 -fopenmp`	`solver-jacobi-mt-O0.exec`
`-O3 -fopenmp`	`solver-jacobi-mt-O3.exec`
`-O3 -march=native -fopenmp`	`solver-jacobi-mt-O3-native.exec`

Compilar com `make`, em cada máquina de execução, e registrar os comandos efetivamente usados e a versão do GCC. Não acrescentar outras opções de otimização nem substituir uma das versões. A opção `-fopenmp`, presente no Makefile, deverá ser mantida em todas elas: habilita o suporte ao OpenMP e não é um nível de otimização.

A opção `-march=native` especializa o código para a máquina de compilação. Portanto, **recompilar essa versão na máquina em que será executada**, inclusive na demonstração; não pressupor que o binário gerado em outra máquina seja compatível. Preservar e comprovar a integridade do fonte e do Makefile.

O relatório deverá explicar as diferenças entre ausência de otimização, otimização `-O3` e especialização `-march=native`, relacionando-as às métricas observadas. Não presumir que a versão mais otimizada será sempre mais rápida, nem afirmar que ocorreu vetorização apenas porque foi usado `-O3`.

2.2 Escolha das três métricas

Neste trabalho, o termo métrica abrange uma contagem de evento ou uma medida derivada de eventos do `perf`. O **tempo de relógio será obrigatório adicionalmente às três métricas escolhidas**; não poderá substituir uma delas.

Consultar `perf list` nas duas máquinas e realizar uma coleta preliminar para confirmar que as três métricas são efetivamente mensuráveis. Exemplos discutidos na seção 3 de `main.pdf` incluem ciclos, instruções aposentadas, IPC/CPI, faltas de cache, faltas de TLB e erros de predição de desvios. São exemplos, não uma seleção obrigatória.

Para cada métrica, documentar:

1. Nome conceitual, evento(s) ou nome usado com `-M`, unidade e significado arquitetural.

2. Hipótese investigada e justificativa de sua relevância para Jacobi e para as otimizações.
3. Comando de coleta, escopo da medição e, quando derivada, fórmula e eventos necessários.
4. Disponibilidade e comparabilidade entre as máquinas. Nomes iguais não garantem semântica idêntica; explicar diferenças e limitações da *Performance Monitoring Unit* — PMU (Unidade de Monitoramento de Desempenho).

IPC e CPI são inversos e não deverão ser apresentados como duas escolhas independentes. Métricas derivadas deverão ser calculadas por repetição, a partir de eventos coletados na mesma execução. Não tratar `<not supported>` ou `<not counted>` como zero.

2.3 OpenMP e controle das condições

O grupo poderá escolher e justificar **uma quantidade fixa de threads**, maior que um e compatível com as duas máquinas, mantendo-a em todos os experimentos comparativos. Registrar `OMP\_NUM\_THREADS`, `OMP\_DYNAMIC` e as configurações de afinidade, como `OMP\_PROC\_BIND`, `OMP\_PLACES` ou `taskset`, quando utilizadas. Verificar se o runtime respeita a quantidade solicitada e evitar sobreposição excessiva de threads sobre as CPUs disponíveis.

Registrar a identificação das máquinas, modelo e geração da CPU, núcleos físicos e CPUs lógicas, tamanhos de cache, memória instalada, sistema operacional, kernel, GCC, runtime OpenMP e versão do `perf`. Controlar ou registrar carga concorrente, afinidade, frequência dinâmica, Turbo e condições térmicas. Em processadores híbridos, identificar os tipos de núcleo utilizados e as PMUs correspondentes.

2.4 Entradas, validação e repetições

Utilizar os cinco tamanhos indicados e manter os mesmos arquivos, tolerância e parâmetros entre versões e máquinas. Identificar cada instância pela dimensão n, pelos arquivos e pelo volume de dados. Para a matriz densa de coeficientes, são n^2 valores; explicitar se o eixo de quantidade de dados representa n, número de elementos ou bytes.

Antes das medições, verificar a correção numérica, a convergência e a compatibilidade das saídas das três versões. A validação da solução do sistema linear deve comparar o vetor x obtido na execução do código da equipe, arquivo `saida.bin` do exemplo abaixo:

```
./solver-jacobi-mt-O3.exec tolerancia exibir_erro A1000.bin B1000.bin saida.bin
```

com o arquivo `Xnnnn.bin`, onde `nnnn` é a quantidade de variáveis resolvidas. No exemplo, o arquivo que deve ser comparado é `X1000.bin`. Para comparar os arquivo, use o fonte `main-diff-mat.cpp`[2] O arquivo final é **de saída**, não uma entrada com o vetor inicial. Nas medições, usar parâmetro `1` para acompanhar a convergência do método, se for o caso. Exemplo para a instância de dimensão 1000:

```
OMP_NUM_THREADS=4 OMP_DYNAMIC=FALSE perf stat -e cycles \
  ./solver-jacobi-mt-O3.exec 1E-10 0 A1000.bin B1000.bin saida-teste.bin
```

Esse exemplo pressupõe que `cycles` esteja disponível; quatro threads e a tolerância apresentada são ilustrativas, não escolhas obrigatórias.

Realizar **dez execuções independentes para cada combinação de métrica, instância, versão e máquina**. Na coleta separada por métrica, são:

$$3 \text{ métricas} \times 5 \text{ tamanhos} \times 3 \text{ versões} \times 2 \text{ máquinas} \times 10 = 900 \text{ execuções.}$$

Cada execução deverá registrar também o tempo de relógio. Será permitido medir as três métricas conjuntamente nas mesmas dez execuções por combinação de instância, desde que todos os eventos

[2] github: `https://github.com/mzamith-ufrrj/Parallel-Programming-exercises/tree/main/01-CPU-OpenMP-pThread/02-matrizes/matriz.cpp`

necessários sejam coletados de forma válida, sem comprometimento da comparação por multiplexação. Isso não reduz as dez observações exigidas para cada métrica. Se houver coleta separada, identificar a série de tempo associada a cada métrica, sem misturar silenciosamente as séries.

Para contadores limitados, dividir a coleta em conjuntos menores; registrar os percentuais de tempo ativo dos eventos e discutir a multiplexação. Contadores devem abranger as threads OpenMP, não apenas a thread principal. Coletas globais da máquina exigem justificar a atribuição dos eventos à aplicação e considerar interferências.

3 Resultados, gráficos e análise

Para **cada combinação experimental**, apresentar a média aritmética e o **desvio padrão amostral** do tempo e da métrica. Não confundir desvio padrão com erro padrão ou percentual de variação apresentado pela ferramenta. Entregar tabelas consolidadas e dados brutos com identificação da máquina, versão, instância, configuração de threads, métrica, repetição, valor, unidade e tempo.

O relatório deverá conter obrigatoriamente:

- **Gráficos de tempo × quantidade de dados**, comparando as três versões em cada máquina e permitindo comparar a máquina nova com a velha.
- **Gráficos das três métricas × quantidade de dados**, permitindo comparar versões e máquinas; para métricas de escalas ou unidades diferentes, utilizar painéis separados ou apresentação devidamente justificada.
- Médias e barras de erro correspondentes a **um desvio padrão**, eixos com unidades, legendas, identificação das configurações e explicação das escalas utilizadas.
- **Análise interpretativa dos gráficos**, relacionando tempo, variabilidade, tamanho das entradas, otimizações e características arquiteturais. Discutir caches, TLB, memória, instruções, paralelismo e sincronização conforme a pertinência das métricas escolhidas.

Explicar se os resultados sustentam as hipóteses, quais diferenças são consistentes diante da dispersão e quais conclusões os dados não permitem. Não atribuir toda diferença ao clock nem deduzir causalidade exclusivamente de uma correlação. Avaliar o efeito do compilador separadamente das diferenças entre as máquinas. A medição externa por `perf stat` inclui a execução completa, com leitura, alocação, cálculo e gravação; não descrevê-la como tempo exclusivo do núcleo de Jacobi.

4 Material de entrega

O envio pelo SIGAA deverá conter:

- Identificação dos integrante.
- Scripts externos de compilação/execução, coleta, validação, estatística e geração dos gráficos; todos os arquivos necessários para reproduzir o procedimento.
- Dados brutos das repetições, logs do `perf`, registros de validação, tabelas consolidadas e gráficos. Identificar as instâncias e documentar sua obtenção no endereço fornecido.
- `README.md` com ambiente, versões, comandos, parâmetros, métricas, quantidade de threads, afinidade, instruções de reprodução e limitações conhecidas.
- Relatório em PDF com método experimental, justificativas, resultados, gráficos, análise e conclusões. Capturas de tela sem interpretação não substituem a explicação.
- `USO.md`, inclusive quando não houver uso dessas ferramentas.

Verificar a reprodução do procedimento antes da entrega. Preparar o ambiente de demonstração com os scripts enviados pelo SIGAA. A recompilação necessária na máquina de apresentação não constitui nova oportunidade de entrega nem autoriza mudanças no material.

5 Declaração de uso de LLM

Se não houver uso de LLM, registrar expressamente essa informação. Se houver, informar:

1. Ferramenta, modelo ou versão quando disponível e integrantes que a utilizaram.
2. Onde foi utilizada: estudo, planejamento, scripts externos, testes, processamento estatístico, gráficos ou revisão; identificar os artefatos afetados.
3. Como foi utilizada: objetivos, prompts e respostas relevantes, e o que foi aceito, adaptado ou rejeitado.
4. Como as sugestões foram verificadas no fonte original, na documentação, nos dados e nos testes, incluindo erros e correções.

Não é necessário anexar todo o histórico, mas os registros devem permitir relacionar o auxílio ao trabalho entregue. Remover senhas, tokens e dados pessoais desnecessários. Uma declaração genérica como "usamos IA" não atende ao requisito. O uso declarado não será penalizado; grupos que não utilizarem LLM poderão obter a pontuação integral com uma declaração clara. A permissão de uso de LLM **não autoriza alterar o resolvedor**.

Durante a arguição, não será permitido consultar LLM nem receber ajuda de outros integrantes. O discente poderá consultar o código e o material entregue para fundamentar suas respostas.

6 Demonstração com instância inédita e arquitetura híbrida

> **Na apresentação, os integrantes serão arguidos com uma instância totalmente diferente das utilizadas nos experimentos, fornecida pelo docente no momento da avaliação. A execução ocorrerá em uma máquina com processador Intel(R) Core(TM) Ultra 7 265KF e 128 GB de memória RAM. O programa será executado primeiro apenas nos P-cores e, depois, apenas nos E-cores. Todos os integrantes deverão estudar essa arquitetura, pois as perguntas estarão relacionadas à máquina de execução e ao comportamento observado.**

Segundo a ficha oficial da Intel, o Core Ultra 7 265KF pertence à família Core Ultra Série 2, codinome Arrow Lake, e apresenta:

Característica	Especificação
Núcleos e threads	20 núcleos: 8 P-cores e 12 E-cores; 20 threads; sem Hyper-Threading.
Frequência base	3,9 GHz nos P-cores; 3,3 GHz nos E-cores.
Frequência turbo	Até 5,4 GHz nos P-cores; até 4,6 GHz nos E-cores; máximo de 5,5 GHz com Turbo Boost Max 3.0.
Cache	30 MB de Intel Smart Cache (L3); 36 MB de L2 no total.
Memória suportada	DDR5 até 6400 MT/s, dois canais. A máquina da avaliação tem 128 GB instalados.
Potência	Potência base de 125 W; potência turbo máxima de 250 W.
Recursos pertinentes	Intel Thread Director; extensões SSE4.1, SSE4.2 e AVX2.

As frequências máximas são limites especificados, não garantias de frequência sustentada em todos os núcleos. A velocidade e a organização da RAM efetivamente instalada deverão ser verificadas na máquina, não inferidas do limite suportado pelo processador.

O grupo deverá saber identificar os conjuntos de CPUs correspondentes aos P-cores e aos E-cores e aplicar afinidade de modo que **todas as threads** permaneçam no conjunto selecionado, sem misturar os tipos de núcleo. Quando disponíveis, consultar:

```
lscpu
cat /sys/bus/event_source/devices/cpu_core/cpus
cat /sys/bus/event_source/devices/cpu_atom/cpus
```

```
perf list
```

Não presumir que os P-cores sejam as CPUs 0–7 ou que os E-cores sejam as CPUs seguintes; confirmar a topologia no Linux. Os nomes `cpu_core` e `cpu\_atom` identificam PMUs distintas, quando expostas pelo sistema; `cpu\_atom` não significa que o processador inteiro seja um Intel Atom.

Saber selecionar eventos apropriados a cada PMU e explicar incompatibilidades ou limitações. A demonstração incluirá as três versões compiladas e o procedimento de coleta; o docente definirá a instância e os testes executados no tempo disponível. Não é exigida previamente uma nova campanha completa de dez repetições sobre a instância inédita.

7 Avaliação oral e barema

Cada discente responderá a **seis perguntas principais**, formuladas pelo docente no dia. Os eixos orientadores são:

1. **Execução de instruções:** pipeline, execução supersescalar e fora de ordem, ciclos, instruções aposentadas, IPC/CPI e relação com o tempo.
2. **Hierarquia de memória:** caches, localidade, capacidade, largura de banda, latência e efeito do tamanho das matrizes.
3. **TLB e endereçamento:** tradução de endereços, páginas, faltas de TLB e distinção entre faltas de TLB, cache e página.
4. **Otimizações:** diferenças entre `-O0`, `-O3` e `-march=native`, SIMD/AVX2, vetorização e preservação da correção numérica.
5. **Arquitetura híbrida e OpenMP:** P-cores e E-cores do Core Ultra 7 265KF, afinidade, PMUs, threads, ausência de Hyper-Threading nesse modelo, barreiras e custos de sincronização.
6. **Análise experimental:** escolha das métricas, repetições, média, desvio padrão, multiplexação, interpretação dos gráficos e previsão/explicação do comportamento da instância inédita na máquina da avaliação.

Quando houver uso de LLM, o docente também poderá solicitar a explicação de uma sugestão e de sua validação. Os eixos não são um questionário para decorar: as perguntas estarão relacionadas ao fonte, às otimizações, aos resultados e à máquina de execução. Pedidos de esclarecimento integram a mesma pergunta e não acrescentam itens de pontuação.

A nota será **individual, de 0 a 10**. As evidências técnicas podem ser comuns ao grupo, mas as respostas orais serão avaliadas separadamente. São preservados os pesos do trabalho de Sistemas Operacionais, adaptando-se o conteúdo dos critérios técnicos:

Critério	Máximo
Execução correta do Jacobi original; três versões do Makefile com OpenMP; três métricas válidas e justificadas; comparação nas duas máquinas e integridade dos fontes	2,0
Testes reproduzíveis: cinco tamanhos, dez repetições por combinação, validação numérica, média, desvio padrão, gráficos e análise	1,0
Documentação, fontes completos, scripts, dados e reprodução do ambiente	0,5
Declaração de uso ou não uso de LLM e, quando aplicável, rastreabilidade e validação das sugestões	0,5
Seis perguntas orais individuais (1,0 por pergunta)	6,0
Total	**10,0**

A arguição corresponde a **60% da nota**. Uma entrega técnica com pontuação máxima soma apenas 4,0 sem as respostas orais. O trabalho é uma avaliação e corresponde a **1/3 da nota final**.

8 Referências e material de apoio

- `main.pdf`, seção 3: “Perf interface”, especialmente repetição e variabilidade, CPUs híbridas, afinidade, métricas, otimizações e cuidados experimentais.
- Fonte, Makefile e instâncias: `http://www.dcc.ufrrj.br/~marcelo/aulas/TN712/fontes.zip`.
- Ficha oficial Intel do Core Ultra 7 265KF (link), consultada em 12/10/2026.
- Documentação local: `man perf-stat`, `man perf-list` e `man taskset`.
- GCC, opções de otimização: `https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html`.
- GCC: runtime OpenMP e variáveis de ambiente (link).